

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Homework 2

NOTE: You are not required to hand in the following exercises, but you are strongly encouraged to complete them to strengthen your understanding of the concepts covered in class.

1. Consider the following C code snippet. Assume all `fork()` calls are successful.

```
void foo() {
    int val = 10;
    if (fork() == 0) {
        val += 5;
        if (fork() == 0) {
            val *= 2;
        } else {
            wait(NULL);
            val -= 2;
        }
    } else {
        wait(NULL);
        val += 2;
    }
    printf("PID %d: val = %d\n", getpid(), val);
    exit(0);
}
```

- A. Draw the process tree created by this function.
 - B. List every possible value of `val` that will be printed to the console.
 - C. Explain why the parent process's `val` is not affected by the child's modifications.
2. Suppose a process forks n children to perform parallel tasks. It must print "All tasks complete" only after every child has finished.
 - A. Write a code fragment using a loop and `wait()` that ensures the parent reaps all N children before proceeding.
 - B. Define a *zombie* process. What specific action (reaping) must the parent take to move the child from a zombie state to being fully discarded by the kernel?
 - C. Describe the mechanism Linux uses to prevent children from remaining zombies indefinitely if the parent process terminates without reaping.

- **Welcome: Sean**

- [LogOut](#)

